

A CVODES corrector convergence failure in a cluster-dynamics right-hand side

Notes for a collaborator integrating SUNDIALS into MoDELib, from the equivalent failure in the ZrMicro slow step.

The message

```
[ERROR][rank 0][.../src/cvodes/cvodes.c:8201][cvHandleFailure]
At t = 13627.1158326934 and h = 0.00142070583632146,
the corrector convergence test failed repeatedly or with |h| = hmin.
```

This is CCode returning CV_CONV_FAILURE, **flag -4**. The “corrector” is the Newton iteration that solves the implicit BDF stage; CVODES tried it, failed, cut the step, tried again, and ran out of retries.

Two flags are easy to confuse with it and mean something different:

flag	name	meaning
-1	CV_TOO_MUCH_WORK	mxstep exhausted — it is <i>grinding</i> , not failing
-3	CV_ERR_FAILURE	the error test failed repeatedly — an accuracy problem, not a Newton one
-4	CV_CONV_FAILURE	this one
-8	CV_RHSFUNC_FAIL	your RHS returned an unrecoverable error
-15	CV_CONSTR_FAIL	CCodeSetConstraints was violated

Because -15 is distinct, **a positivity constraint vector is not what produced this message**. And because the default hmin is 0, unless you called CCodeSetMinStep explicitly it is the “*failed repeatedly*” half of the sentence that applies, not the |h| = hmin half. The message text covers both cases; only one of them is yours.

The observation that narrows it fastest

The Newton iteration matrix is

$$M = I - \gamma J, \quad \gamma \propto h$$

so as $h \rightarrow 0$, $M \rightarrow I$, and **Newton converges trivially on any smooth right-hand side**. CVODES responds to a corrector failure by cutting h. If cutting h does not fix it, the assumption behind that response has been violated.

So a corrector failure at small h is almost never “the problem is too stiff.” It means one of exactly two things:

1. **J is not the Jacobian of f** — the iteration is solving the wrong linear system, and no h repairs that; or
2. **f is not smooth in y** at that point — a kink, a clamp, a branch, a NaN, or a derivative that diverges.

That single argument removes most of the search space, and it is why loosening tolerances and hoping usually does not work here.

Ranked causes, with the test for each

1. An analytic Jacobian inconsistent with the residual

By far the most common cause, and the cheapest to rule out.

Test: comment out `CVodeSetJacFn` and let CVODES build its own difference-quotient Jacobian. If the integration then completes, your Jacobian is wrong. Thirty seconds, and it is conclusive either way.

A difference-quotient Jacobian is slower but *always consistent with whatever f you actually wrote*, which is exactly the property you want while diagnosing.

If it turns out to be the Jacobian, difference it properly rather than reading it. Our `jac_check.cpp`, under `ZrMicro/cpp_utils/`, compares a forward-AD Jacobian against a central difference of the **same** residual. Two details in it are worth copying:

- **difference each column with a relative step** $h_j = \text{eps_rel} \cdot |y_j|$. Our state spans ~ 34 decades (concentrations $\sim 1e-20$ up to $\rho_N \sim 1e14$) and no single absolute step serves all of it;
- **report, but do not fail on, components sitting exactly on a floor.** The floor makes the residual non-differentiable there, so AD returns a one-sided derivative while a central difference straddles the kink. That is a property of the model, not a bug in the differentiation — and confusing the two will send you hunting for a week.

2. A non-smooth right-hand side

`max(x, 0)`, `fabs`, concentration clamps, floors, `if (c > 0) ... else ...` — anything that makes f non-differentiable *inside* the residual. Newton sees a kink, and reducing h never smooths it.

Cluster-dynamics right-hand sides are full of these. Ours has `max(drdt, 0)` in the coalescence rate.

A clamp applied inside f is different from a clamp applied to the state between steps. The second is a legitimate operator-split choice; the first is a discontinuity handed to an implicit integrator that assumes it is differentiating something.

3. A species crossing zero, or its floor

Test: print the whole state vector at the failure and look for a component that is negative, NaN, or sitting at $\sim 1e-30$.

This was our case, and it was *deterministic* — the same point failed on its own, twice, across two separate invocations. It was not an extreme-value point either: 30th–50th percentile in every mobile

species. What singled it out was its **location**, 51 b from a Dirichlet face inside the boundary layer, where the gradient is steepest.

4. A sqrt of a population that is emptying

$d(\sqrt{x})/dx \rightarrow \infty$ as $x \rightarrow 0$, so Jacobian entries genuinely diverge as a family depopulates. Our sink strength carries $\sqrt{(n_k - c_k)}$, and loop radii carry $\sqrt{(\text{content}/\text{number})}$.

This is a real mechanism rather than a rounding artifact, and it compounds with cause 3: the same event that takes a population to its floor is the one that makes its Jacobian row blow up.

5. Mixed units

Importing SUNDIALS into MoDELib means choosing between SI and MoDELib's normalized units, and the choice has to be made **once, for the whole state vector**. Mixed units give $|J|$ entries spanning ranges that no tolerance setting can serve, and the symptom is exactly this one.

Worth an explicit audit if the integration is new rather than inherited.

$t = 13627$ s is itself evidence

It integrated *fine* up to there. That rules out uniform stiffness and says something happens at that time.

Convert it to dose at your generation rate — at $1e-7$ dpa/s it is $1.4e-3$ dpa, which lands in the nucleation transient, where concentrations move by decades and populations first appear. Then ask what crosses a threshold there: a loop family nucleating, a source term switching on, a concentration reaching its floor, a reaction channel becoming active.

$h/t \approx 1e-7$ also tells you the step collapsed rather than drifted down, which is consistent with an event rather than with gradually worsening conditioning.

What fixed it for us — note the direction

Counterintuitively, **tightening**.

On node 41804 of the 500 nm march, `rtol` alone changed nothing and `atol` alone changed nothing. `rtol = 1e-8` **together with** `atol = 1e-30` integrated the point in 0.083 s. The $\langle c \rangle$ loop content was crossing the positivity floor, and the tighter tolerance makes the integrator approach the crossing with a properly resolved step history instead of taking one large step that overshoots into a region where the residual is evaluated on a negative argument.

We checked that the answer was the physical continuation and not a numerical one: the point's five nearest neighbors fall by a factor of 277 over the same substep, so the whole region is collapsing and this point merely reaches the floor first.

Tighter, never looser. Loosening also “works,” and it is the wrong trade — it buys convergence by accepting a less accurate answer at the one point in the domain already known to be difficult.

Two caveats, because this is a measurement and not a law:

- the **classical** advice runs the other way — if a component's `atol` sits *below* its numerical noise floor, its error weight $1/(rtol \cdot |y| + atol)$ becomes enormous and no iterate can satisfy the test, and the fix is to **raise** `atol` for that component. Both are worth trying; ours is what we measured on this system;
- per-component tolerances (`CCodeSVtolerances`) are the right instrument when the state mixes concentrations with cumulative accumulators. We give the accumulators an `atol` so large that their error weight is numerically zero, which keeps them from controlling a step size they have no business controlling.

Instrumentation worth adding before guessing again

```
long nni, ncf, netf, nst, nje;
CCodeGetNumNonlinSolvIters(mem, &nni);
CCodeGetNumNonlinSolvConvFails(mem, &ncf); /* corrector */
CCodeGetNumErrTestFails(mem, &netf);      /* error test */
CCodeGetNumSteps(mem, &nst);
CCodeGetNumJacEvals(mem, &nje);
```

Read them together:

pattern	reading
ncf large, netf small netf large, ncf small	it really is Newton — causes 1–2 above an accuracy problem, not a Newton one; you would normally see –3
nni/nst climbing toward the iteration cap nst near mxstep	the iteration matrix is poorly conditioned or stale you are grinding, and would get –1 rather than –4

Also print `y` at the failure. The failure time and step size are already in the message; the state is the part you need and the part CVODES cannot guess for you.

Treat the failure as retryable, not fatal

Two lessons from running this over ~90 000 points per substep.

A point that fails inside a batch may succeed alone. The batch step is not deterministic: a per-thread CVODE workspace is reused across cases, and under dynamic scheduling which case follows which varies from run to run. A point sitting near the solver's failure boundary is then decided by scheduling rather than by its own state. Re-running just the failures repacks them into a small batch, pairs them differently, and usually clears them. On the 500 nm march one point of 90 617 failed, and the identical step — same checkpoint, same inputs reproduced to every printed digit — completed with zero failures when re-run.

A point that fails alone, twice, is real. It needs a solver or a model answer, not another attempt. That distinction is the whole value of retrying: a retry that always succeeds tells you nothing, and one that never does tells you where to look.

One thing to be careful of if you add a tolerance for failures: substituting an identity step (freezing the point for that substep) does not *recover* the point. It declares the march successful without it. That is sometimes the right engineering call, but it should be a recorded decision rather than a silent default.

A related failure mode worth pre-empting: a point that neither converges nor fails, but exhausts `mxstep` and returns `-1` after a very long time. We measured single substeps taking 5h47m and 3.0 h against a normal 57 s, from roughly 700 such points in a 189 533-node field. Lowering `CVodeSet-NumSteps` converts a hang into a failure, which is the one thing a retry path can actually see.

If you do only one thing

Switch off the analytic Jacobian and re-run. It is the cheapest test, it addresses the most likely cause, and its result is informative whichever way it comes out.

Version note

The behavior above was measured with SUNDIALS 7.1.1; none of this touches API that has changed since. The path in the error message is simply SUNDIALS' own error handler reporting its build location, and carries no information about the fault.